



K. R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Lab File

Operating System Lab

(ENCS 351)

Submitted by

Abhishek Kumar [2301010322]

Btech CSE core

Section -E

Submitted to

Ms. Reenu Batra

Associate Professor

School of Engineering & Technology

Sohna, Haryana 122103, India

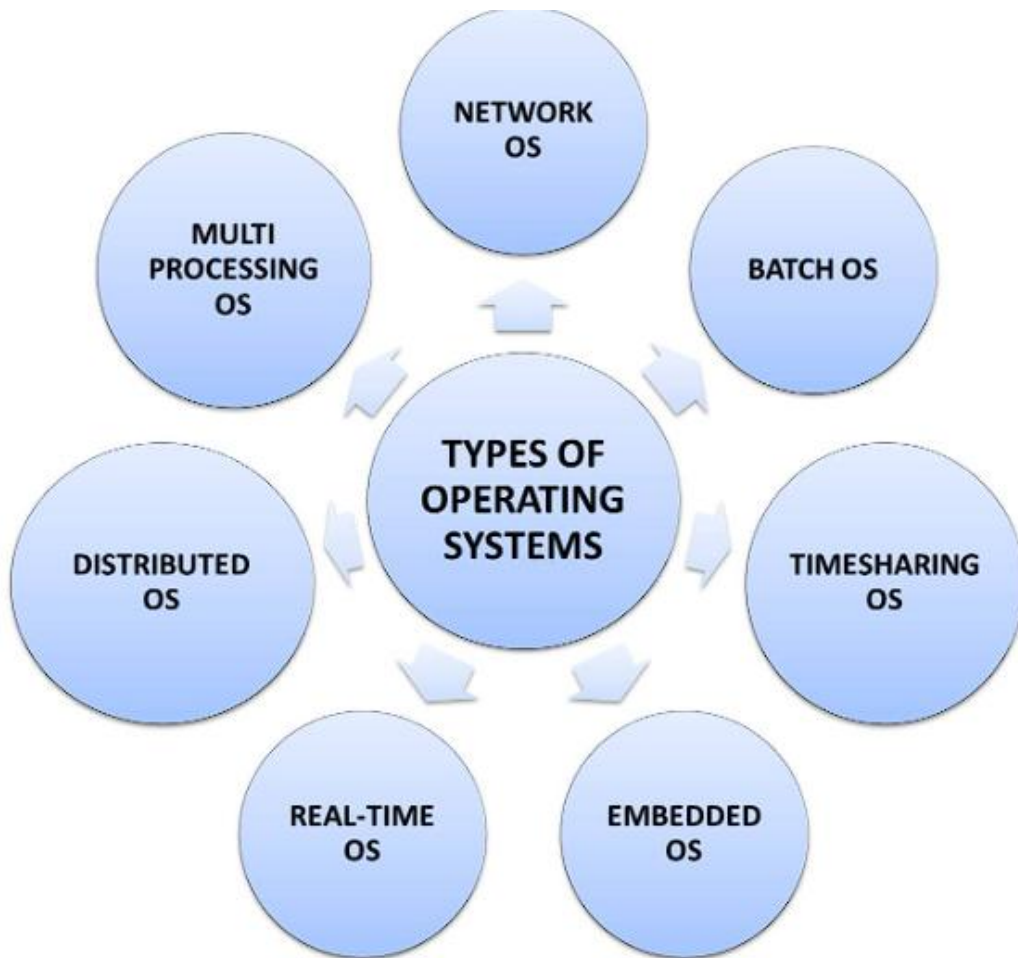
S. NO	TITLE	PAGE NO	REMARK
1	Different Types of Operating System	3-10	
2	MS-DOS Commands	11-17	
3	CPU Scheduling Algorithm: First Come First Serve	18-19	
4	CPU Scheduling Algorithm: Shortest Job First	20-21	
5	CPU Scheduling Algorithm: Round Robin	22-24	
6	CPU Scheduling Algorithm: Priority Scheduling	24-25	
7	Unix Commands	26-31	
8	Producer and Consumer's Problem	30-33	
9	Read and Write Problem	34-35	
10	Banker's Problem	36-39	

EXPERIMENT-1

Aim :- Write about different types of OS system.

Introduction

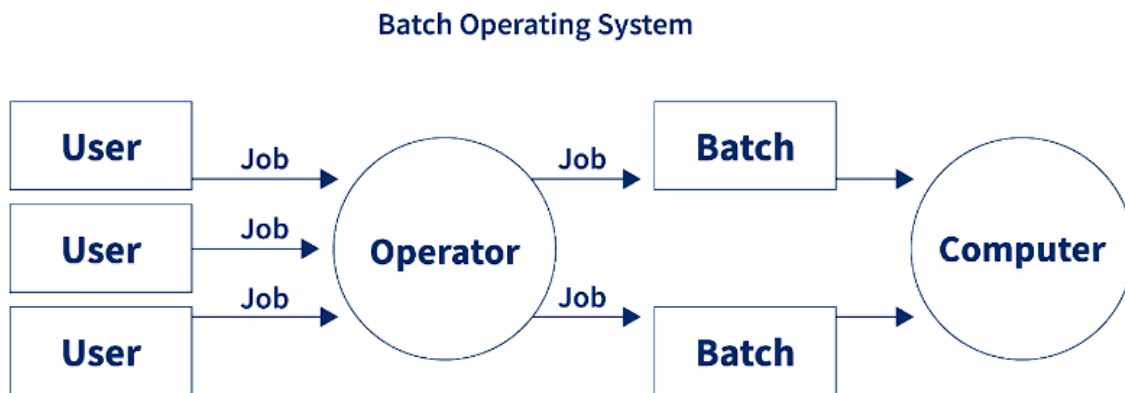
Definition: An operating system (OS) is the most crucial software that runs on a computer. It manages the computer's memory, processes, and all of its hardware and software. It also allows you to communicate with the computer without knowing how to speak the computer's language.



1. Batch Operating System

Definition

A Batch Operating System groups similar tasks or jobs together and executes them automatically in batches. Users submit their jobs to the system, and the OS processes them one by one without any direct interaction during execution.



Advantages

- **High efficiency for repetitive tasks:** Best for jobs like payroll, billing, or large data processing.
- **Reduces idle CPU time:** Jobs are scheduled efficiently, ensuring maximum CPU usage.

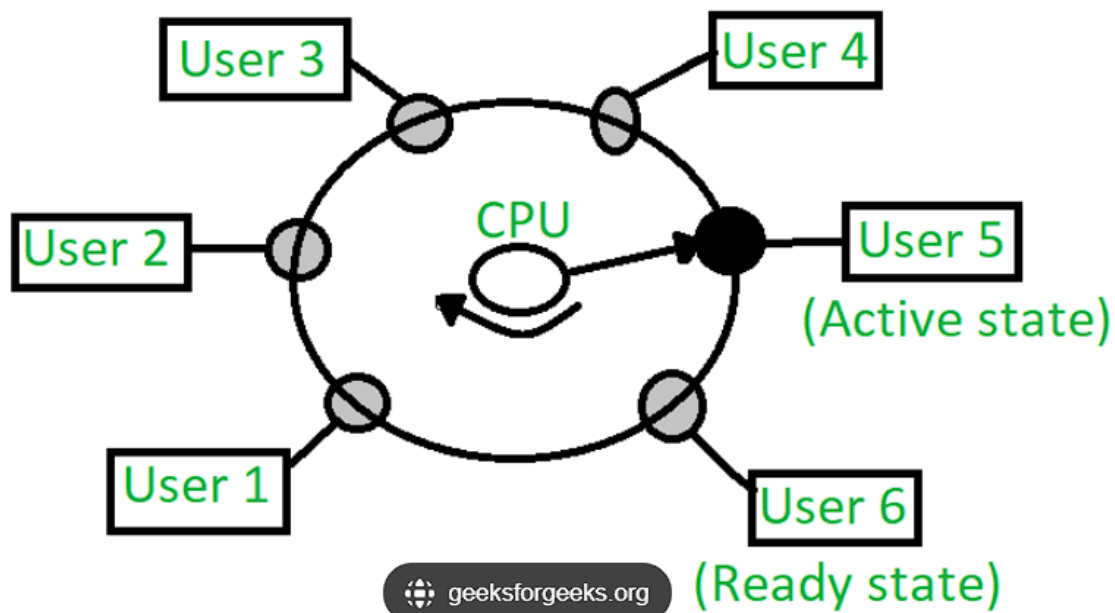
Disadvantages

- **No real-time interaction:** Users cannot interact with the job while it is executing.
- **Debugging is difficult:** If an error occurs, the job may need to be re-run from the beginning.

2. Time-Sharing Operating System

Definition

A time-sharing OS allows multiple users to use a computer system at the same time. It provides each user a small time slice and quickly switches between them, giving the illusion of parallel execution.



Advantages

- **Fast response time:** Users get quick feedback from the system.
- **Efficient CPU utilization:** CPU is shared efficiently among all users.
- **Supports many users:** Ideal for institutes, organizations, and multi-user environments.

Disadvantages

- **Security and privacy concerns:** Multiple users share the system simultaneously.
- **High complexity:** Requires advanced scheduling algorithms.
- **Risk of system overload:** Too many users can slow down the system.

3. Distributed Operating System

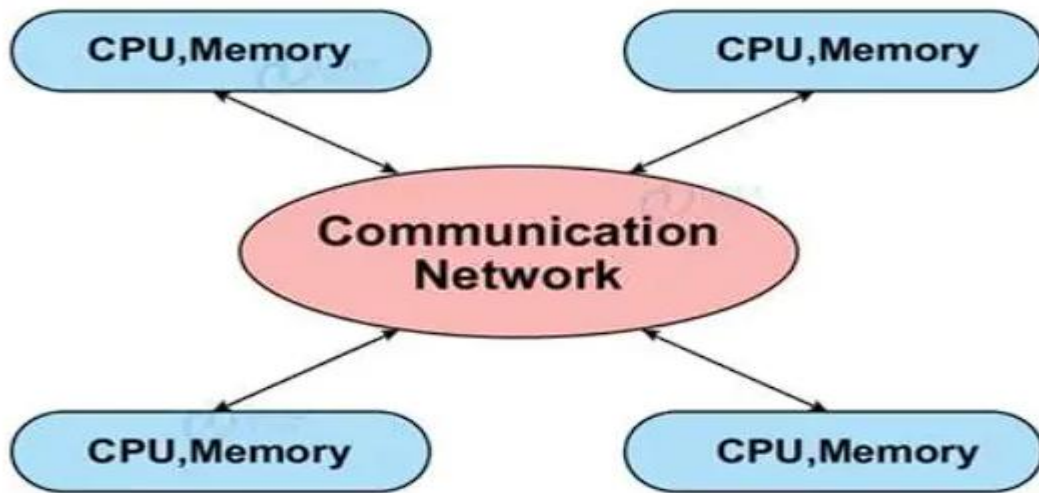
Definition

A Distributed OS manages a group of interconnected computers and makes them appear as a single, unified system. Resources are shared across the network, and tasks may run on different machines.

Advantages

- **High reliability:** If one machine fails, others continue working.
- **Parallel processing:** Tasks can be divided among multiple systems for fast execution.
- **Resource sharing:** Users can share printers, storage, and data easily.

Distributed Operating System



Disadvantages

- **Very complex design:** Synchronization and communication must be managed carefully.
- **Dependency on network:** A network failure can disturb the entire system.
- **Security issues:** Data moving across systems needs strong protection.

4. Network Operating System (NOS)

Definition

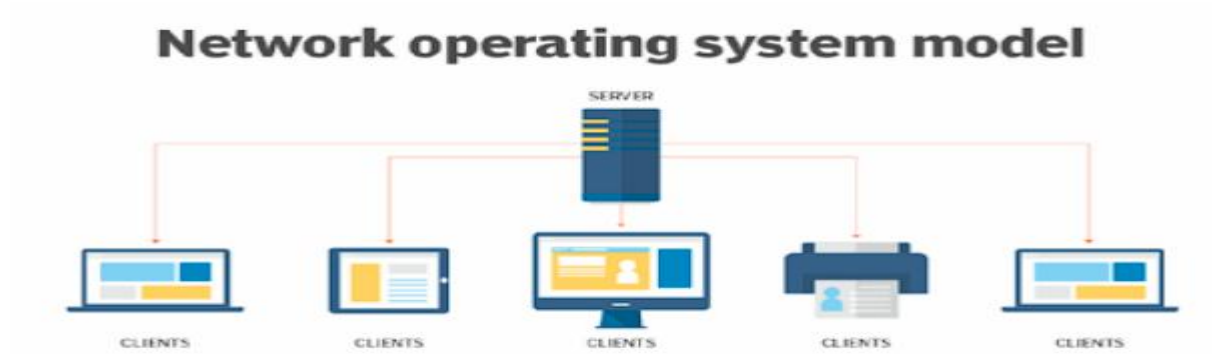
An OS designed to manage network resources such as file sharing, printer sharing, communication, and centralized user management. Examples include Windows Server, UNIX, and Linux server versions.

Advantages

- **Centralized management:** Admin can manage users, security, and devices from one server.
- **Easy communication:** Enables file sharing, messaging, and resource access across computers.

Disadvantages

- **Requires skilled administrators:** Managing networks needs expertise.
- **Expensive setup:** Servers and licenses cost more.
- **Server dependency:** If the central server fails, the entire network may stop.



5.

Real-Time Operating System (RTOS)

Definition

An RTOS is designed to process data and respond within a strict time limit. It is used in situations where delays cannot be tolerated, such as in medical systems, aircraft, robots, and automotive systems.

Advantages

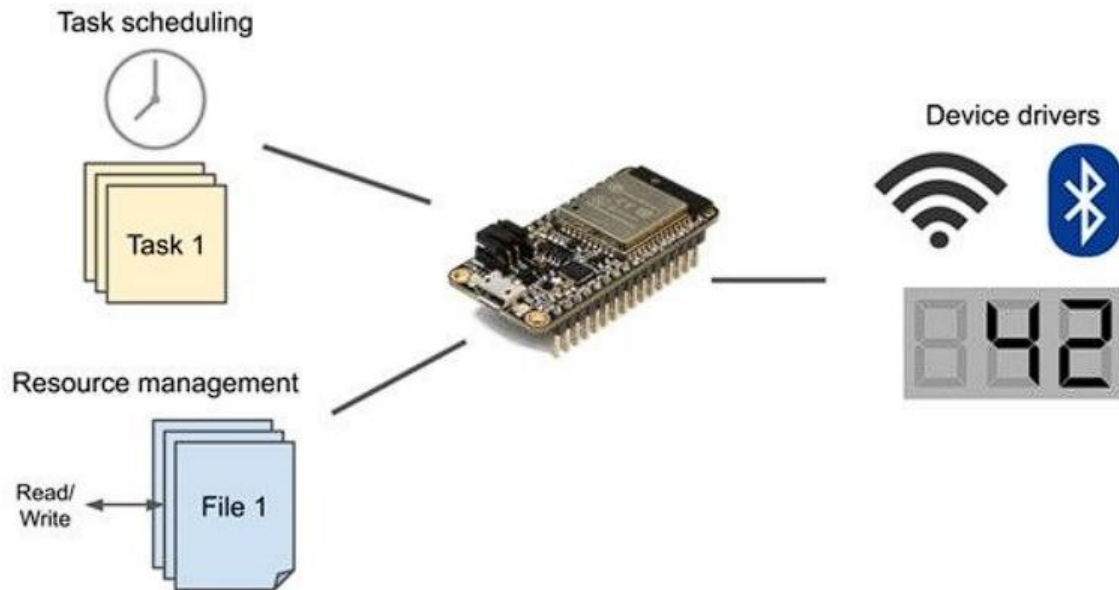
- **Predictable and fast response:** Timely execution of tasks is guaranteed.
- **High reliability:** Ideal for critical systems where failure is not acceptable.
- **Efficient resource utilization:** Handles small tasks very quickly.

Disadvantages

- **Very costly:** Used only in specialized industries.
- **Limited multitasking:** Focuses only on time-critical tasks.

- **Complex design:** Scheduling and priority management are difficult.

Real-Time Operating System (RTOS)



6. Multiprocessing Operating System

Definition

A multiprocessing OS supports more than one processor. All processors work together to execute tasks faster, improving overall system performance.

Advantages

- **High speed and performance:** Tasks are divided and processed simultaneously.
- **Increased reliability:** If one processor fails, others continue working.
- **Better throughput:** More jobs completed in less time.

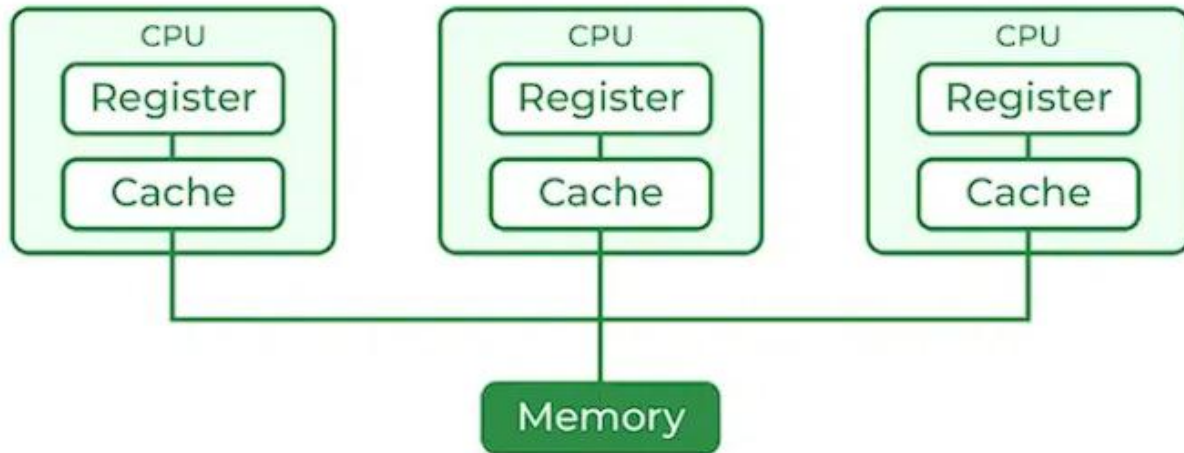
Disadvantages

- **Expensive hardware:** Requires multi-core processors and proper configuration.
- **Complex management:** Scheduling tasks across processors is challenging.

- **Requires compatible software:** Not all applications support multiprocessing.

Multiprocessing

7.



Multi-Programming Operating System

Definition

In a multiprogramming OS, several programs are loaded into memory at once and the OS switches between them to utilize CPU time efficiently.

Advantages

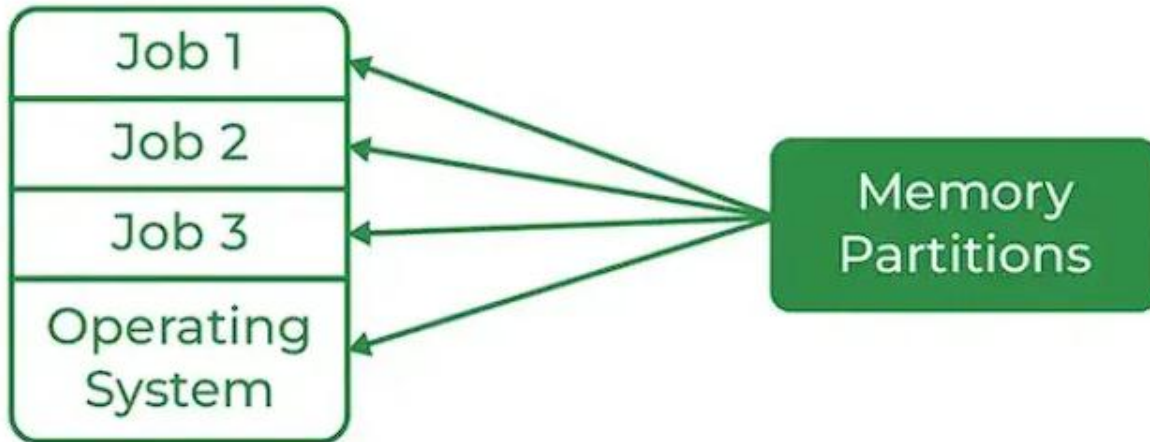
- **Reduces CPU idle time:** CPU is always busy by switching between programs.
- **Increases system throughput:** More processes are completed overall.

Disadvantages

- **Needs more memory:** Multiple programs must be present in memory.
- **Complex memory management:** Requires advanced algorithms.
- **Possibility of deadlocks:** Multiple programs may compete for resources

Multiprogramming

8.



Mobile Operating System

Definition

A Mobile OS is specifically designed for smartphones and tablets. Examples include Android, iOS, KaiOS, and HarmonyOS.

Advantages

- **Touch-optimized interface:** Easy to use for touch-based devices.
- **Energy-efficient:** Optimized for battery-powered devices.
- **Portable and lightweight:** Uses fewer resources compared to desktop OS.

Disadvantages

- **Limited hardware access:** Apps cannot access everything for security reasons.
- **Less customizable (esp. iOS):** Users cannot change system-level features.
- **Security risks:** Downloading apps from unknown sources may expose data.

EXPERIMENT-2

AIM :- To know about MS-Dos Command

1. Command - CD , Syntax- CD [drive:][path]Change, Description -gives the current directory to the specified folder.

```
C:\Users\adity>cd  
C:\Users\adity
```

2. Command - DIR , Syntax- DIR [drive:][path],Description -Displays a list of files and subdirectories

```
Volume in drive C is Windows-SSD  
Volume Serial Number is 2652-1589  
  
Directory of C:\Users\adity  
  
19-11-2025  11:04    <DIR>          .  
11-11-2025  23:39    <DIR>          ..  
13-09-2025  10:04    <DIR>          .anaconda  
10-08-2025  15:09           176 .bash_history  
21-10-2025  23:10    <DIR>          .conda  
13-09-2025  10:05           25 .condarc  
13-09-2025  10:04    <DIR>          .continuum  
15-11-2025  12:21    <DIR>          .cursor  
26-07-2025  23:48           202 .gitconfig  
20-11-2025  23:18    <DIR>          .gk  
13-09-2025  10:22    <DIR>          .ipython  
13-09-2025  14:12    <DIR>          .jupyter  
23-02-2025  00:30    <DIR>          .lc  
20-09-2025  20:29           20 .lessht  
07-02-2025  13:57           115 .node_repl_history  
31-05-2025  12:52    <DIR>          .openshot_qt  
10-08-2025  23:51    <DIR>          .rest-client  
27-03-2025  20:42    <DIR>          .thumbnails  
27-09-2025  00:26    <DIR>          .VirtualBox  
29-01-2025  23:26    <DIR>          .vscode  
13-09-2025  10:59    <DIR>          anaconda3
```

Abhishek Kumar

2301010322

Section - E

3. Command -COPY, Syntax-COPY [source] [destination],Description -Copies files from one location to another.

4.Command - DEL , Syntax - DEL [drive:][path] , Description - Deletes one or more files.

```
user@computer:~$ del report_backup.docx
1 file(s) deleted.

user@computer:~$ dir
Volume in drive C is OS
Volume Serial Number is 1234-5678

Directory of C:\Users\user

09/17/2025  01:57 PM    <DIR>          Documents
09/17/2025  01:57 PM    <DIR>          Downloads
09/17/2025  01:57 PM    <DIR>          Desktop
09/16/2025  11:22 AM                12,345  report.docx
09/15/2025  03:10 PM            567,890  photo.jpg
09/17/2025  01:56 PM                234    script.py
               3 File(s)            580,469 bytes
               3 Dir(s)      12,345,678,901 bytes free

user@computer:~$
```

5.Command-REN, Syntax- REN [drive:][path][filename1] [filename2],
Description - Renames a file or directory.

```
user@computer:~$ ren script.py my_first_script.py

user@computer:~$ dir
Volume in drive C is OS
Volume Serial Number is 1234-5678

Directory of C:\Users\user

09/17/2025  01:57 PM    <DIR>          Documents
09/17/2025  01:57 PM    <DIR>          Downloads
09/17/2025  01:57 PM    <DIR>          Desktop
09/16/2025  11:22 AM                12,345  report.docx
09/15/2025  03:10 PM            567,890  photo.jpg
09/17/2025  01:56 PM                234  my_first_script.py
               3 File(s)            580,469 bytes
               3 Dir(s)      12,345,678,901 bytes free

user@computer:~$
```

6. Command - MKDIR, SyntaxMKDIR [drive:][path], Description -Creates a new directory.

```

08-11-2025 13:06 30,331,110 icpc_obs_windows.zip
10-02-2025 12:36 <DIR> js
12-11-2025 00:34 <DIR> Links
12-11-2025 00:34 <DIR> Music
25-11-2025 15:15 <DIR> new_folder
12-11-2025 13:59 <DIR> OneDrive
06-02-2025 15:37 84 package-lock.json

```

7. Command - RMDIR, Syntax- RMDIR [drive:][path], Description -Removes an existing directory.

```

Volume in drive C is Windows-SSD
Volume Serial Number is 2652-1589

Directory of C:\Users\adity

19-11-2025 11:04 <DIR> .
11-11-2025 23:39 <DIR> ..
13-09-2025 10:04 <DIR> .anaconda
10-08-2025 15:09 176 .bash_history
21-10-2025 23:10 <DIR> .conda
13-09-2025 10:05 25 .condarc
13-09-2025 10:04 <DIR> .continuum
15-11-2025 12:21 <DIR> .cursor
26-07-2025 23:48 202 .gitconfig
20-11-2025 23:18 <DIR> .gk
13-09-2025 10:22 <DIR> .ipython
13-09-2025 14:12 <DIR> .jupyter
23-02-2025 00:30 <DIR> .lc
20-09-2025 20:29 20 .lessht
07-02-2025 13:57 115 .node_repl_history
31-05-2025 12:52 <DIR> .openshot_qt
10-08-2025 23:51 <DIR> .rest-client
27-03-2025 20:42 <DIR> .thumbnails
27-09-2025 00:26 <DIR> .VirtualBox
29-01-2025 23:26 <DIR> .vscode
13-09-2025 10:59 <DIR> anaconda3

```

Abhishek Kumar

2301010322

Section - E

8. Command - TYPE, Syntax- TYPE [drive:][path], Description -filename Displays the contents of a text file.

```
user@computer:~$ type my_first_script.py
# This is a simple Python script
# It prints a welcome message to the console

print("Hello, world! This is my first script.")

user@computer:~$
```

9. Command - EDIT, Syntax- EDIT [drive:][path]filename, Description -Opens the MS-DOS text editor for editing a specified file.

```
user@computer:~$ edit my_first_script.py
```

10. Command - CHKDSK, Syntax-CHKDSK [volume:][[path]filename] [/F] [/V] [/R] [/X] [/I] [/C] [/L[:size]] [/B] , Description -Scans and fixes errors on a disk.

```
user@computer:~$ CHKDSK
The type of the file system is NTFS.
Volume label is OS.

CHKDSK is verifying files (stage 1 of 3)...
 1344 file records processed.
File verification completed.
CHKDSK is verifying indexes (stage 2 of 3)...
 2315 index entries processed.
Index verification completed.
CHKDSK is verifying security descriptors (stage 3 of 3)...
 1344 security descriptors processed.
Security descriptor verification completed.
Windows has scanned the file system and found no problems.

user@computer:~$
```

Abhishek Kumar

2301010322

Section - E

11. Command - FORMAT, Syntax- FORMAT volume [/FS:file-system] [/V:label] [/Q] [/L[:size]] [/A:size] [/C] [/X], Description - Prepares a storage medium for data storage.

```
user@computer:~$ FORMAT E: /FS:NTFS /V:DATA_DRIVE /Q
The type of the file system is RAW.
The new file system is NTFS.

WARNING, ALL DATA ON NON-REMOVABLE DISK
DRIVE E: WILL BE LOST!
Proceed with Format (Y/N)? Y

Formatting 1.0 GB
QuickFormatting...
Format complete.
Volume label: DATA_DRIVE
500 GB total disk space.
498 GB available on disk.

Volume Serial Number is 1234-5678

user@computer:~$
```

12. Command - XCOPY, Syntax- XCOPY [source] [destination] [/E] [/C] [/H] [/R] [/Y], Description - Copies files and directories, including subdirectories.

```
user@computer:~$ cd ..

user@computer:~$ xcopy project_files backup /E /I
Does backup specify a file name or directory name on the target
(F = file, D = directory)? D
project_files\photo.jpg
1 File(s) copied

user@computer:~$ dir backup
Volume in drive C is OS
Volume Serial Number is 1234-5678

Directory of C:\Users\user\backup

09/17/2025  02:15 PM    <DIR>          .
09/17/2025  02:15 PM    <DIR>          ..
09/15/2025  03:10 PM                567,890  photo.jpg
                1 File(s)                567,890 bytes
                2 Dir(s)            12,345,678,901 bytes free

user@computer:~$
```

13. Command -TREE, Syntax- TREE [drive:][path], Description -Graphically displays the folder structure of a drive or path.

```
user@computer:~$ tree
Folder PATH listing for volume OS
Volume serial number is 1234-5678
C:\Users\user
|---backup
|   |---photo.jpg
|---Desktop
|---Documents
|---Downloads
|---my_first_script.py
|---photo.jpg
|---report.docx

No subdirectories were found.

user@computer:~$
```

Abhishek Kumar

2301010322

Section - E

14. Command -DATE , Syntax- DATE [MM-DD-YYYY] Description -Displays or sets the system date.

```
C:\Users\adity>date
The current date is: 25-11-2025
```

15. Command -TIME , Syntax- TIME [HH: MM: SS], Description -Displays or sets the system time.

```
user@hostname:~$ date +%T
12:30:55
```

16. Command - HELP, Syntax-HELP [command], Description - Provides help information for MS-DOS commands.

```
user@hostname:~$ help cd
cd: cd [-L|[-P [-e]] [-@]] [dir]
    Change the shell working directory.

    Change the current directory to DIR. The default DIR is the value of the
    HOME shell variable.

    The variable CDPATH defines the search path for the directory containing
    DIR. Alternative directory names in CDPATH are separated by a colon (:).
    A null directory name is the same as the current directory. If DIR begins
    with a slash (/), CDPATH is not used.

    Options:
    -L    force symbolic links to be followed
    -P    force symbolic links to be resolved to the physical directory
           before changing directory
    -e    if the -P option is supplied, and the current working directory
           cannot be determined successfully, exit with a non-zero status
    -@    on systems that support it, display a filename preceded by an
           at sign (@)

    Exit Status:
    Returns 0 if the directory is changed, non-zero otherwise.
```

17. Command -EXIT , Syntax- EXIT, Description -Exits the MS-DOS command prompt or a batch file.

```
user@hostname:~$ echo "Hello, World!"
Hello, World!
user@hostname:~$ exit
logout
[Process completed]
```

18. Command -ATTRIB , Syntax- ATTRIB [+ R|-R] [+A|-A] [+ H|-H] [+ S|-S] [d:][path]filename [/S],
Description -Sets or clears file attributes (Read-Only, Archive, System, Hidden), managing file
visibility and access in MS-DOS

```
C:\Users\User>attrib +H my_document.txt
C:\Users\User>dir /a:h
Volume in drive C has no label.
Volume Serial Number is XXXX-XXXX

Directory of C:\Users\User

26/09/2025  12:18 PM                1024 my_document.txt
               1 File(s)                1,024 bytes
               0 Dir(s)   XXX,XXX,XXX bytes free
```

19. Command -MODE , Syntax-MODE [device] [BAUD=b] [PARITY=p] [DATA=d] [STOP=s] ,
Description -Configures system devices.

```
C:\Users\User>MODE COM1 BAUD=96 PARITY=N DATA=8 STOP=1
COM1: 9600, n, 8, 1, ...
C:\Users\User>
```

20. Command - DISKCOPY, Syntax- DISKCOPY [drive1:][path1][filename1]
[drive2:][path2][filename2], Description -Copies the contents of one disk to another.

```
C:\Users\User>DISKCOPY A: A:
Insert SOURCE diskette in drive A:
Press any key to continue . . .
Copying 80 tracks, 18 sectors per track, 2 side(s) . . .
Insert TARGET diskette in drive A:
Press any key to continue . . .
Copy complete.
Copy another diskette (Y/N)?N
```

EXPERIMENT -3

AIM:- To write a c program to simulate the CPU scheduling algorithm First Come First Serve (FCFS).

CODE :- FIRST COME FIRST SERVE

```
import java.util.Scanner;

public class FCFS {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int[] bt = new int[20];
        int[] wt = new int[20];
        int[] tat = new int[20];

        float wtavg, tatavg;
        int n;

        System.out.print("Enter the number of processes: ");
        n = sc.nextInt();

        for (int i = 0; i < n; i++) {
            System.out.print("Enter Burst Time for Process " + i + ": ");
            bt[i] = sc.nextInt();
        }

        wt[0] = 0;
        wtavg = 0;
        tat[0] = bt[0];
        tatavg = tat[0];

        for (int i = 1; i < n; i++) {
            wt[i] = wt[i - 1] + bt[i - 1];
            tat[i] = wt[i] + bt[i];

            wtavg += wt[i];
            tatavg += tat[i];
        }
    }
}
```

```
System.out.println("\nPROCESS\tBURST TIME\tWAITING TIME\tTURNAROUND TIME");
for (int i = 0; i < n; i++) {
    System.out.println("P" + i + "\t\t" + bt[i] + "\t\t" + wt[i] + "\t\t" + tat[i]);
}

System.out.println("\nAverage Waiting Time = " + (wtavg / n));
System.out.println("Average Turnaround Time = " + (tatavg / n));

sc.close();
}
}
```

OUTPUT:-

```
Enter the number of processes: 3
Enter Burst Time for Process 0: 24
Enter Burst Time for Process 1: 3
Enter Burst Time for Process 2: 5
```

PROCESS	BURST TIME	WAITING TIME	TURNAROUND TIME
P0	24	0	24
P1	3	24	27
P2	5	27	32

```
Average Waiting Time = 17.0
Average Turnaround Time = 27.666666
```

EXPERIMENT - 4

AIM:- To write a program to stimulate the CPU scheduling algorithm

Shortest job first (Non- Preemption)

CODE :- SHORTEST JOB FIRST

```
import java.util.Scanner;
public class SJF {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] p = new int[20]; // process IDs
        int[] bt = new int[20]; // burst times
        int[] wt = new int[20]; // waiting times
        int[] tat = new int[20]; // turnaround times
        int n;
        float wtavg = 0, tatavg = 0;
        System.out.print("Enter the number of processes: ");
        n = sc.nextInt();

        for (int i = 0; i < n; i++) {
            p[i] = i;
            System.out.print("Enter Burst Time for Process " + i + ": ");
            bt[i] = sc.nextInt();
        }

        // Sort processes by burst time (simple bubble sort)
        for (int i = 0; i < n; i++) {
            for (int k = i + 1; k < n; k++) {
                if (bt[i] > bt[k]) {
                    // swap burst times
                    int temp = bt[i];
                    bt[i] = bt[k];
                    bt[k] = temp;

                    // swap process IDs
                    temp = p[i];
                    p[i] = p[k];
                    p[k] = temp;
                }
            }
        }
    }
}
```

```
        wt[0] = 0;
        tat[0] = bt[0];
        tatavg = tat[0];

        for (int i = 1; i < n; i++) {
            wt[i] = wt[i - 1] + bt[i - 1];
            tat[i] = wt[i] + bt[i];

            wtavg += wt[i];
            tatavg += tat[i];
        }

        System.out.println("\nPROCESS\tBURST TIME\tWAITING TIME\tTURNAROUND TIME");
        for (int i = 0; i < n; i++) {
            System.out.println("P" + p[i] + "\t\t" + bt[i] + "\t\t" + wt[i] + "\t\t" + tat[i]);
        }

        System.out.println("\nAverage Waiting Time = " + (wtavg / n));
        System.out.println("Average Turnaround Time = " + (tatavg / n));

        sc.close();
    }
}
```

OUTPUT:-

```
Enter the number of processes: 3
Enter Burst Time for Process 0: 65
Enter Burst Time for Process 1: 2
Enter Burst Time for Process 2: 7
```

PROCESS	BURST TIME	WAITING TIME	TURNAROUND TIME
P1	2	0	2
P2	7	2	9
P0	65	9	74

```
Average Waiting Time = 3.666667
Average Turnaround Time = 28.333334
```

EXPERIMENT - 5

AIM:- To simulate the CPU scheduling algorithm round-robin.

CODE :- ROUND ROBIN

```
public class RoundRobinExact {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n, t;
        int[] bt = new int[10]; // burst times
        int[] ct = new int[10]; // copy of burst times
        int[] wt = new int[10]; // waiting times
        int[] tat = new int[10]; // turnaround times
        boolean[] finished = new boolean[10];

        double awt = 0, att = 0; // use double for precision
        int time = 0;

        System.out.print("Enter the no of processes -- ");
        n = sc.nextInt();

        for (int i = 0; i < n; i++) {
            System.out.print("Enter Burst Time for process " + (i + 1) + " -- ");
            bt[i] = sc.nextInt();
            ct[i] = bt[i];
            finished[i] = false;
        }
        System.out.print("Enter the size of time slice -- ");
        t = sc.nextInt();

        int remaining = n;
        while (remaining > 0) {
            for (int i = 0; i < n; i++) {
                if (!finished[i]) {
                    if (bt[i] <= t) {
                        time += bt[i];
                        tat[i] = time; // completion time
                        bt[i] = 0;
                        finished[i] = true;
                        remaining--;
                    }
                }
            }
        }
    }
}
```

```
        } else {
            bt[i] -= t;
            time += t;
        }
    }
}

// Calculate totals
for (int i = 0; i < n; i++) {
    wt[i] = tat[i] - ct[i];
    att += tat[i];
    awt += wt[i];
}
System.out.printf("\nThe Average Turnaround time is -- %.6f", (att / n));
System.out.printf("\nThe Average Waiting time is -- %.6f\n", (awt / n));
System.out.println("\nPROCESS\tBURST TIME\tWAITING TIME\tTURNAROUND TIME");
for (int i = 0; i < n; i++) {
    System.out.println((i + 1) + "\t\t" + ct[i] + "\t\t" + wt[i] + "\t\t" + tat[i]);
}
sc.close();
```

OUTPUT:-

```
Enter the no of processes -- 3
Enter Burst Time for process 1 -- 45
Enter Burst Time for process 2 -- 6
Enter Burst Time for process 3 -- 7
Enter the size of time slice -- 3
```

```
The Average Turnaround time is -- 31.666667
```

```
The Average Waiting time is -- 12.333333
```

PROCESS	BURST TIME	WAITING TIME	TURNAROUND TIME
1	45	13	58
2	6	9	15
3	7	15	22

EXPERIMENT - 6

AIM:- To write a c program to simulate the CPU scheduling priority algorithm.

CODE :-

```
import java.util.Scanner;
public class PriorityScheduling {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int[] p = new int[20]; // process IDs
        int[] bt = new int[20]; // burst times
        int[] pri = new int[20]; // priorities
        int[] wt = new int[20]; // waiting times
        int[] tat = new int[20]; // turnaround times
        int n;

        float wtavg = 0, tatavg = 0;
        System.out.print("Enter the number of processes: ");
        n = sc.nextInt();

        for (int i = 0; i < n; i++) {
            p[i] = i;
            System.out.print("Enter the Burst Time & Priority of Process " + i + ": ");
            bt[i] = sc.nextInt();
            pri[i] = sc.nextInt();
        }

        // Sort by priority (lower number = higher priority)
        for (int i = 0; i < n; i++) {
            for (int k = i + 1; k < n; k++) {
                if (pri[i] > pri[k]) {
                    // swap priorities
                    int temp = pri[i];
                    pri[i] = pri[k];
                    pri[k] = temp;

                    // swap burst times
                    temp = bt[i];
                    bt[i] = bt[k];
                    bt[k] = temp;
                }
            }
        }
    }
}
```

```

        // swap process IDs
        temp = p[i];
        p[i] = p[k];
        p[k] = temp;
    }
}
wt[0] = 0;
tat[0] = bt[0];
tatavg = tat[0];
for (int i = 1; i < n; i++) {
    wt[i] = wt[i - 1] + bt[i - 1];
    tat[i] = wt[i] + bt[i];
    wtavg += wt[i];
    tatavg += tat[i];
}
System.out.println("\nPROCESS\tPRIORITY\tBURST TIME\tWAITING TIME\tTURNAROUND TIME");
for (int i = 0; i < n; i++) {
    System.out.println("P" + p[i] + "\t\t" + pri[i] + "\t\t" + bt[i] + "\t\t" + wt[i] + "\t\t" + tat[i]);
}

System.out.println("\nAverage Waiting Time = " + (wtavg / n));
System.out.println("Average Turnaround Time = " + (tatavg / n));
sc.close();
}

```

OUTPUT:-

```

Enter the number of processes: 3
Enter the Burst Time & Priority of Process 0: 14
3
Enter the Burst Time & Priority of Process 1: 12 2
Enter the Burst Time & Priority of Process 2: 4 1

PROCESS PRIORITY      BURST TIME      WAITING TIME      TURNAROUND TIME
P2          1          4          0          4
P1          2          12         4          16
P0          3          14         16         30

Average Waiting Time = 6.6666665
Average Turnaround Time = 16.666666

```

EXPERIMENT - 7

AIM:- To know about Unix Commands.

1. **Command -cd , Syntax-** cd Documents, **Description** -Changes the current working directory.

```
user@hostname:~$ cd downloads
user@hostname:~/downloads$
```

2. **Command -ls , Syntax-** ls, **Description** -Lists files and directories in the current directory.

```
terminal@terminal-temple ~ $ ls
Documents      Downloads      Music          Pictures
```

3. **Command -pwd , Syntax-** pwd, **Description** -print the current working directory.

```
terminal@terminal-temple Downloads $ pwd
/home/terminal/Downloads
terminal@terminal-temple Downloads $
```

4. **Command -mkdir , Syntax-** mkdir new_folder, **Description** -Creates a new directory.

```
terminal@terminal-temple Downloads $ mkdir myFolder
terminal@terminal-temple Downloads $ ls
myFolder
```

1. **Command -rmdir , Syntax-**rmdir empty_folder, **Description -**
Removes an empty directory.

```
user@hostname:~$ ls -F
downloads/  documents/  empty_folder/
user@hostname:~$ rmdir empty_folder
user@hostname:~$ ls -F
downloads/  documents/
user@hostname:~$
```

2. **Command - mv , Syntax-** mv file.txt Documents/, **Description -**Moves files or directories.

```
user@hostname:~$ ls -F
documents/  old_report.txt
user@hostname:~$ mv old_report.txt new_report.txt
user@hostname:~$ ls -F
documents/  new_report.txt
user@hostname:~$
```

3. **Command -touch , Syntax-** touch new_file.txt, **Description -**Creates an empty file or updates the access and modification times.

```
terminal@terminal-temple Downloads $ touch myfile.txt
terminal@terminal-temple Downloads $ ls
myfile.txt      myFolder
terminal@terminal-temple Downloads $
```

4. **Command -cp , Syntax-** cp file1.txt file2.txt, **Description** -Copies files or directories.

```
terminal@terminal-temple Downloads $ cp file.txt myFile.txt
terminal@terminal-temple Downloads $ ls
file.txt          myfile.txt        myFile.txt        myFolder
terminal@terminal-temple Downloads $
```

5. **Command - mv , Syntax-** mv file1.txt file2.txt,, **Description** -Move files or directories.

```
terminal@terminal-temple Downloads $ mv file.txt myfile.txt
terminal@terminal-temple Downloads $ ls
myfile.txt        myFile.txt        myFolder
terminal@terminal-temple Downloads $
```

6. **Command -pwd , Syntax-** rm old_file.txt, **Description** -Remove files or directories.

```
terminal@terminal-temple Downloads $ rm myFile.txt
terminal@terminal-temple Downloads $ ls
myfile.txt        myFolder
terminal@terminal-temple Downloads $
```

- 7..**Command -chmod , Syntax-** chmod 644 file.txt, **Description** - Changes the

```
user@hostname:~$ ls -l myscript.sh
-rw-r--r-- 1 user group 1024 Oct 1
10:00 myscript.sh
user@hostname:~$ chmod 755
myscript.sh
user@hostname:~$ ls -l myscript.sh
-rwxr-xr-x 1 user group 1024 Oct 1
10:00 myscript.sh
user@hostname:~$
```

7.-Changes the owner and group of a file or directory.

```
user@hostname:~$ ls -l website.html
-rw-r--r-- 1 olduser webdevs 5421 Oct 5 11:00 website.html

user@hostname:~$ sudo chown
newuser:prod website.html

[sudo] password for user:

user@hostname:~$ ls -l website.html
-rw-r--r-- 1 newuser prod 5421 Oct 5 11:00 website.html

user@hostname:~$
```

8. **Command - ln, Syntax-** ln -s target_file symlink, **Description** - Create links

```
user@hostname:~$ ls -l original_file.txt
-rw-r--r-- 1 user group 100 Oct 10 12:00 original_file.txt

user@hostname:~$ ln original_file.txt
hard_link.txt

user@hostname:~$ ls -l original_file.txt
hard_link.txt
-rw-r--r-- 2 user group 100 Oct 10 12:00 original_file.txt
-rw-r--r-- 2 user group 100 Oct 10 12:00 hard_link.txt

user@hostname:~$
```

9. **Command - cat, Syntax-** cat file1.txt file2.txt, **Description** - Concatenates files and display their contents.

```
user@hostname:~$ cat daily_notes.txt
- Review client proposal
- Send updated budget to team lead
- Schedule Friday's demo
user@hostname:~$
```

10. **Command - head, Syntax-** head file.txt, **Description -** Display the first few lines of a file.

```
user@hostname:~$ head -n 5 log_data.txt
[Line 1: 2025-11-09 INFO Starting server... ]
[Line 2: 2025-11-09 DEBUG Loading config... ]
[Line 3: 2025-11-09 INFO User 'admin' logged in ]
[Line 4: 2025-11-09 INFO Connecting to database ]
[Line 5: 2025-11-09 DEBUG Query successful ]
user@hostname:~$
```

11. **Command - tail , Syntax-** tail file.txt, **Description -** Display the last few lines of a file.

```
user@hostname:~$ tail -f /var/log/syslog
... (displays the last 10 lines initially)
Nov 9 16:30:01 hostname kernel: [315.000] eth0: link up, 100Mbps, full-duplex
Nov 9 16:30:05 hostname sshd[1234]: Accepted publickey for user from 10.1.1.2 port
Nov 9 16:30:10 hostname CRON[5678]: (root) CMD (command -v d-i && d-i install /usr
<-- This line will only appear after the file grows -->
Nov 9 16:30:30 hostname systemd[1]: Reached target User and Group Name Lookups.
<-- This line will appear seconds later, as the system writes it to the log -->
```

12. **Command -more , Syntax-** more file.txt, **Description -** Displays the contents of a file page by

13.

```
user@hostname:~$ more release_notes.txt
[Line 1: Major Release 2.5.0 - November 2025]
[Line 2: New Features:]
[Line 3: -----]
[Line 4: - Implemented multi-region cloud deployment.]
[Line 5: - Added dark mode UI theme option.]
[Line 6: - Enhanced database query optimization.]
[Line 7: Bug Fixes:]
[Line 8: -----]
[Line 9: - Fixed memory leak in caching service (Bug #1234).]
[Line 10: - Corrected rendering issue on mobile devices.]
[Line 11: - Resolved intermittent API timeout error.]
[Line 12: Known Issues:]
[Line 13: -----]
[Line 14: - Printing feature currently unstable in Chrome.]
[Line 15: - Documentation PDF links are broken.]
--More--(15%)
```

14. **Command - less , Syntax- less file1.txt, Description -**Display the contents of a file with advanced navigation features.

```
user@hostname:~$ less install_guide.md
# Installation Guide: Service V3.0

## Prerequisites

1.  **Operating System:** Any modern Linux distribution (Ubuntu, CentOS, RHEL).
2.  **Memory:** Minimum 4GB RAM.
3.  **Storage:** Minimum 50GB free disk space.

## Step-by-Step Installation

1.  **Download and Extract:**
    Use `wget` to fetch the tarball:
    `$ wget https://example.com/service-v3.tar.gz`
    `$ tar -xzf service-v3.tar.gz`

2.  **Configure Environment:**
    Navigate to the extracted directory:
    `$ cd service-v3.0`
    Edit the main configuration file:
    `$ nano config/main.conf`

3.  **Run Setup Script:**
    Make the setup script executable and run it:
    `$ chmod +x setup.sh`
    `$ ./setup.sh`

install_guide.md
```

15. **Command -diff , Syntax- diff file1.txt file2.txt, Description -** Compare files line by line.

```
user@hostname:~$ diff config_v1.txt config_v2.txt
2c2
< port = 8080
---
> port = 9000
4a5
> log_file = /var/log/app.log
user@hostname:~$
```

16. **Command -patch , Syntax- patch file.txt < change.diff, Description -** Applies a diff file to update a target file.

```
user@hostname:~$ ls -l config_v1.txt
-rw-r--r-- 1 user group 100 Nov 8 10:00 config_v1.txt
user@hostname:~$ patch config_v1.txt config.patch
patching file config_v1.txt
user@hostname:~$ diff config_v1.txt config_v2.txt
user@hostname:~$
```

EXPERIMENT – 8

AIM:- Producer and consumer's problem.

CODE:-

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty; // counts empty slots
sem_t full; // counts filled slots
pthread_mutex_t mutex; // mutual exclusion

void* producer(void* arg) {
    int item = 1;

    while (1) {
        sem_wait(&empty); // wait if buffer is full
        pthread_mutex_lock(&mutex);

        printf("Producer produced item %d\n", item);
        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        item++;

        pthread_mutex_unlock(&mutex);
        sem_post(&full); // signal that buffer has data
        sleep(1);
    }
}

void* consumer(void* arg) {
    while (1) {
        sem_wait(&full); // wait if buffer is empty
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumer consumed item %d\n", item);
```

```
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);    // signal that buffer has empty space
        sleep(2);
    }
}

int main() {
    pthread_t prod, cons;

    // Initialize semaphores
    sem_init(&empty, 0, BUFFER_SIZE); // buffer initially empty
    sem_init(&full, 0, 0);           // buffer initially has 0 items
    pthread_mutex_init(&mutex, NULL);

    // Create threads
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    // Wait for threads
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    // Clean up
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

OUTPUT :-

```
Producer produced item 1
Consumer consumed item 1
Producer produced item 2
Consumer consumed item 2
Producer produced item 3
Producer produced item 4
Consumer consumed item 3
Producer produced item 5
Producer produced item 6
Consumer consumed item 4
Producer produced item 7
Producer produced item 8
Consumer consumed item 5
Producer produced item 9
Producer produced item 10
Consumer consumed item 6
Producer produced item 11
Consumer consumed item 7
Producer produced item 12
Consumer consumed item 8
Producer produced item 13
Consumer consumed item 9
Producer produced item 14
Consumer consumed item 10
```

EXPERIMENT-9

AIM:-Reader and writer problem.

Code:-

```
#include <stdio.h>
sem_t mutex;    // for mutual exclusion when updating readcount
sem_t writeblock; // controls writer's access to shared data
int readcount = 0; // number of readers currently reading
void* reader(void* arg) {
    int id = *(int*)arg;

    sem_wait(&mutex);    // lock before updating readcount
    readcount++;
    if (readcount == 1) {
        sem_wait(&writeblock); // block writers when first reader enters
    }
    sem_post(&mutex);    // unlock

    // Reading section
    printf("Reader %d is reading...\n", id);
    sleep(1);

    sem_wait(&mutex);    // lock before updating readcount
    readcount--;
    if (readcount == 0) {
        sem_post(&writeblock); // allow writers when last reader leaves
    }
    sem_post(&mutex);

    return NULL;
}

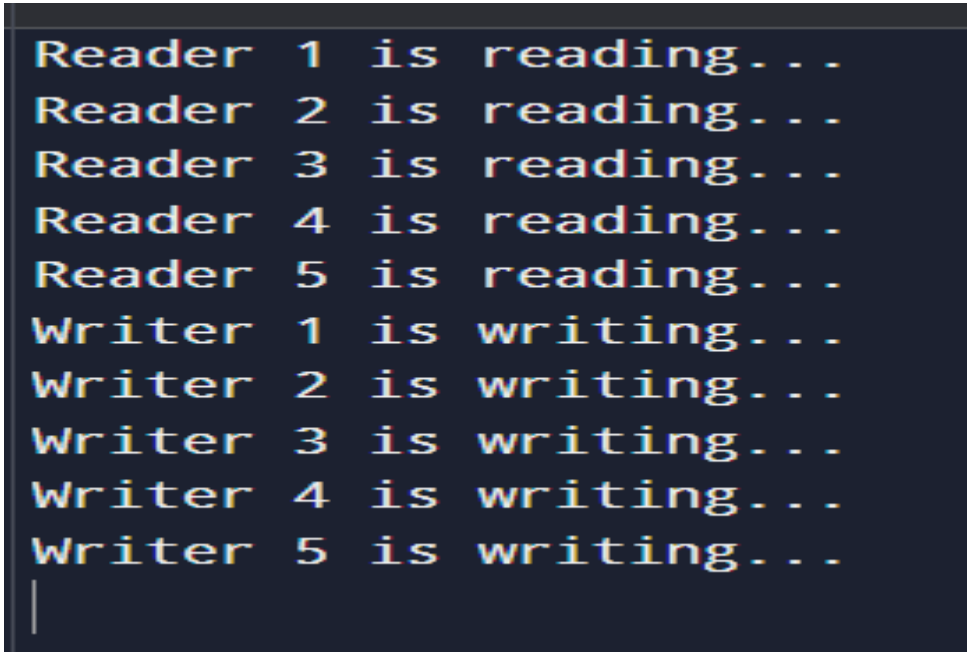
void* writer(void* arg) {
    int id = *(int*)arg;

    sem_wait(&writeblock); // lock writers
    // Writing section
    printf("Writer %d is writing...\n", id);
    sleep(2);
    sem_post(&writeblock); // unlock writers

    return NULL;
}
```

```
int main() {  
    pthread_t r[5], w[5];  
    int id[5];  
  
    sem_init(&mutex, 0, 1);  
    sem_init(&writeblock, 0, 1);  
  
    // Creating reader and writer threads  
    for (int i = 0; i < 5; i++) {  
        id[i] = i + 1;  
        pthread_create(&r[i], NULL, reader, &id[i]);  
        pthread_create(&w[i], NULL, writer, &id[i]);  
    }  
  
    // Join threads  
    for (int i = 0; i < 5; i++) {  
        pthread_join(r[i], NULL);  
        pthread_join(w[i], NULL);  
    }  
  
    sem_destroy(&mutex);  
    sem_destroy(&writeblock);  
  
    return 0;  
}
```

OUTPUT:-



```
Reader 1 is reading...  
Reader 2 is reading...  
Reader 3 is reading...  
Reader 4 is reading...  
Reader 5 is reading...  
Writer 1 is writing...  
Writer 2 is writing...  
Writer 3 is writing...  
Writer 4 is writing...  
Writer 5 is writing...  
|
```

EXPERIMENT – 10

AIM:-Banker's problem.

Code:-

```
#include <stdio.h>

int main() {
    int n, m; // n = number of processes, m = number of resources
    int i, j, k;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    printf("Enter the number of resource types: ");
    scanf("%d", &m);

    int allocation[10][10], max[10][10], need[10][10];
    int available[10];
    int finish[10] = {0};
    int safeSeq[10];

    printf("\nEnter Allocation Matrix (%d x %d):\n", n, m);
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("\nEnter Max Matrix (%d x %d):\n", n, m);
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    printf("\nEnter Available Resources (1 x %d):\n", m);
    for (j = 0; j < m; j++) {
        scanf("%d", &available[j]);
    }
}
```

```
// Calculate Need Matrix = Max - Allocation
printf("\nNeed Matrix:\n");
for (i = 0; i < n; i++) {
    for (j = 0; j < m; j++) {
        need[i][j] = max[i][j] - allocation[i][j];
        printf("%d ", need[i][j]);
    }
    printf("\n");
}

// Safety Algorithm
int count = 0; // count of processes in safe sequence

while (count < n) {
    int found = 0;

    for (i = 0; i < n; i++) {
        // skip if already finished
        if (finish[i] == 0) {
            int canAllocate = 1;

            // check if need <= available for all resources
            for (j = 0; j < m; j++) {
                if (need[i][j] > available[j]) {
                    canAllocate = 0;
                    break;
                }
            }

            // if this process can be allocated
            if (canAllocate) {
                // pretend to allocate and then finish the process
                for (k = 0; k < m; k++) {
                    available[k] += allocation[i][k];
                }

                safeSeq[count] = i;
                count++;
                finish[i] = 1;
                found = 1;
            }
        }
    }
}
```

```
    }

    // if no process could be allocated in this pass, not safe
    if (!found) {
        break;
    }
}

if (count == n) {
    int p;
    printf("\nSystem is in a SAFE STATE.\nSafe sequence: ");
    for (p = 0; p < n; p++) {
        printf("P%d ", safeSeq[p]);
    }
    printf("\n");
} else {
    printf("\nSystem is in an UNSAFE STATE (may lead to deadlock).\n");
}

return 0;
}
```

OUTPUT:-

```
Enter the number of processes: 5
Enter the number of resource types: 3

Enter Allocation Matrix (5 x 3):
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Max Matrix (5 x 3):
7 5 3
3 2 2
9 0 2
4 2 2
5 3 3

Enter Available Resources (1 x 3):
3 3 2

Need Matrix:
7 4 3
1 2 2
6 0 0
2 1 1
5 3 1

System is in a SAFE STATE.
Safe sequence: P1 P3 P4 P0 P2
```